

Manual Técnico – Proyecto1

Angel Emanuel Rodriguez Corado

202404856

Índice

Introducción	3
Objetivos.....	3
Público Objetivo	3
Requisitos técnicos.....	3
Hardware	3
Software	3
Lógica General del Sistema	4
Funcionamiento de los Módulos Principales	4
LexicalAnalyzer.ts	4
Token.ts.....	5
Index.ejs.....	6
PensumParser.ts	7
errores.ejs	8
index.js	9
index.ts	10
analyze.route.ts	11
analyze.controller.ts	11
Automata Finito Determinista AFD	12

Introducción

Este documento describe los componentes técnicos del proyecto desarrollado en TypeScript para analizar un lenguaje específico que permite analizar y ver el pensum de la carrera desde un archivo .plfp. El sistema incluye un analizador léxico basado en un autómata finito determinista (AFD), generación de reportes de tokens, manejo de errores, y visualización del Pensum de la carrera.

Objetivos

- Documentar la lógica de implementación del analizador léxico.
- Explicar la estructura del código fuente.
- Servir como guía técnica para mantenimiento o evolución del sistema.

Público Objetivo

Este manual está dirigido a desarrolladores, docentes y estudiantes que deseen entender o modificar la lógica del proyecto, o que necesiten extender su funcionalidad.

Requisitos técnicos

Hardware:

- Procesador Intel i5 o superior
- Mínimo 4 GB de RAM

Software:

- Node.js 18+
- TypeScript
- Express.js
- EJS
- Navegador web moderno
- Acceso a Internet (para consumir la PokeAPI)

Lógica General del Sistema

1. El usuario carga un archivo .plfp o edita el texto directamente en el editor.
2. Al presionar “Analizar”, el backend ejecuta un analizador léxico personalizado basado en AFD.
3. Se generan:
 - a. Tokens válidos → mostrados en tabla. o Errores léxicos → renderizados en una página HTML.
 - b. Pensum → Se muestra el pensum de la carrera subida en el archivo

Funcionamiento de los Módulos Principales

LexicalAnalyzer.ts

- Implementa el Autómata Finito Determinista (AFD).
- Lee línea por línea y analiza cada carácter.
- Clasifica lexemas en:
 - Palabras reservadas
 - Cadenas (entre comillas)
 - Números (enteros)
 - Símbolos
- Registra tokens válidos y errores léxicos con fila y columna.
- **Se trabajan las palabras reservadas de forma que se analizan desde una lista comparándolas con la entrada en el frontend**

Link del código:

https://github.com/Deltai-cod12/LFP_Proyecto1_202404856.git

Token.ts

- Define interfaces para:
- Token: tipo, valor, fila, columna • Error: carácter, descripción, ubicación • Enumera los tipos de tokens reconocidos.

```
enum Type {
  UNKNOWN,
  PAR_OPEN,      // (
  PAR_CLOSE,     // )
  SEMICOLON,     // ;
  COLON,         // :
  BRACKET_OPEN,  // [
  BRACKET_CLOSE, // ]
  BRACE_OPEN,    // {
  BRACE_CLOSE,   // }
  NUMBER,
  STRING,
  RESERVED_WORD,
  COMMA,        // ,
}

class Token {
  private row: number;
  private column: number;
  private lexeme: string;
  private typeToken: Type;
  private typeTokenString: string;

  constructor(typeToken: Type, lexeme: string, row: number, column: number){
    this.typeToken = typeToken;
    this.typeTokenString = Type[typeToken];
    this.lexeme = lexeme;
    this.row = row;
    this.column = column;
  }

  getRow(): number {
    return this.row;
  }

  getColumn(): number {
    return this.column;
  }

  getLexeme(): string {
    return this.lexeme;
  }

  getType(): Type {
    return this.typeToken;
  }

  getTypeTokenString(): string {
    return this.typeTokenString;
  }
}

export {Token, Type}
```

Index.ejs

Pagina principal donde se muestra la tabla tokens

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Pensum USAC - Análisis Léxico</title>
  <link rel="stylesheet" href="/css/estilos.css">
</head>
<body>
  <header>
    <nav>
      <ul>
        <li><button onclick="location.href='/'">Home</button></li>

        <% if (!errors || errors.length === 0) { %>
          <form id="formPensum" action="/pensum" method="POST" style="display:inline;">
            <input type="hidden" name="txtArea" id="pensumContent">
            <button type="submit">Pensum</button>
          </form>
        <% } %>

        <form id="formErrores" action="/error-report" method="POST" style="display:inline;">
          <input type="hidden" name="erroresJSON" id="erroresJSON" value='<%= JSON.stringify(errors || []) %>'>
          <button type="submit">Reporte de Errores</button>
        </form>

        <li>
          <select id="archivoMenu">
            <option disabled selected>Archivo</option>
            <option value="limpiar">Limpiar Editor</option>
            <option value="cargar">Cargar Archivo</option>
            <option value="guardar">Guardar Archivo</option>
          </select>
        </li>
      </ul>
    </nav>
  </header>

  <main>
    <form action="/analyze" method="POST">
      <section id="editor-container">
        <h2>Editor de Entrada</h2>
        <textarea id="editor" name="txtArea" placeholder="Escribe o carga tu archivo .plfp aquí..."><%= codigo || '' %></textarea>
        <button type="submit">Analizar</button>
        <pre id="resultado"></pre>
      </section>
    </form>

    <section id="tablaTokens-container">
      <h2>Tabla de Tokens</h2>
      <table id="tablaTokens">
        <thead>
          <tr>
            <th>#</th>
            <th>Fila</th>
            <th>Columna</th>
            <th>Lexema</th>
            <th>Token</th>
          </tr>
        </thead>
        <tbody>
          <% if (tokens && tokens.length > 0) { %>
            <% tokens.forEach((token, index) => { %>
              <tr>
                <td><%= index + 1 %></td>
                <td><%= token.fila %></td>
                <td><%= token.columna %></td>
                <td><%= token.lexema %></td>
                <td><%= token.token %></td>
              </tr>
            <% }) %>
          <% } %>
        </tbody>
      </table>
    </section>
  </main>

  <input type="file" id="fileInput" accept=".plfp" style="display:none">
  <script src="/js/index.js"></script>
</body>
</html>
```

PensumParser.ts

Guarda los semestres, cursos y áreas luego de realizar el análisis.

```
interface Curso {
  codigo: string;
  nombre: string;
  area: string;
  prerequisitos: string[];
}

interface Semestre {
  numero: string;
  cursos: Curso[];
}

interface PensumExtraido {
  carrera: string;
  semestres: Semestre[];
}

export function extraerPensum(texto: string): PensumExtraido {
  const carreraMatch = texto.match(/Carrera:\s*"([^"]+)"/);
  const carrera = carreraMatch ? carreraMatch[1] : "Desconocida";

  const semestresEncontrados: Semestre[] = [];
  const semestreStartRegex = /Semestre:\s*(\d+)\s*/g;
  let currentSemestreMatch;
  let lastIndex = 0;

  while ((currentSemestreMatch = semestreStartRegex.exec(texto)) !== null) {
    const semestreNumero = currentSemestreMatch[1];
    const semestreStartIndex = currentSemestreMatch.index + currentSemestreMatch[0].length;
    let braceCount = 1;
    let semestreEndIndex = -1;

    for (let i = semestreStartIndex; i < texto.length; i++) {
      if (texto[i] === '{') braceCount++;
      else if (texto[i] === '}') braceCount--;

      if (braceCount === 0) {
        semestreEndIndex = i;
        break;
      }
    }

    if (semestreEndIndex !== -1) {
      const cuerpoSemestre = texto.substring(semestreStartIndex, semestreEndIndex);
      const cursos: Curso[] = [];

      const cursoRegex = /Curso:\s*(\d+)\s*{([^\s}]*?)}/g;
      let cursoMatch;
      while ((cursoMatch = cursoRegex.exec(cuerpoSemestre)) !== null) {
        const codigo = cursoMatch[1];
        const cuerpoCurso = cursoMatch[2];

        const nombreMatch = cuerpoCurso.match(/Nombre:\s*"([^"]+)"/);
        const areaMatch = cuerpoCurso.match(/Area:\s*(\d+)/);
        const prerequisitosMatch = cuerpoCurso.match(/Prerequisitos:\s*"([^\"]*)"\/);

        const nombre = nombreMatch ? nombreMatch[1] : "Sin nombre";
        const area = areaMatch ? areaMatch[1] : "0";
        const prerequisitos = prerequisitosMatch
          ? prerequisitosMatch[1].split(',').map(p => p.trim()).filter(p => p !== '')
          : [];

        cursos.push({ codigo, nombre, area, prerequisitos });
      }

      semestresEncontrados.push({ numero: semestreNumero, cursos });
      semestreStartRegex.lastIndex = semestreEndIndex + 1;
    } else {
      console.error(`Error: Semestre ${semestreNumero} no tiene llave de cierre.`);
      break;
    }
  }

  return { carrera, semestres: semestresEncontrados };
}
```

errores.ejs

Se muestra en el navegador los errores que hubieron durante el análisis

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8" />
  <title>Reporte de Errores Léxicos</title>
  <link rel="stylesheet" href="/css/estilos.css" />
</head>
<body>
  <header>
    <nav>
      <ul>
        <li><button onclick="location.href='/'>Home</button></li>
        <li><button onclick="location.href='/error-report'">Error Report</button></li>
        <li>
          <select id="archivomenu" disabled>
            <option disabled selected>Archivo</option>
          </select>
        </li>
      </ul>
    </nav>
  </header>

  <main>
    <section id="reporteErrores-container">
      <h2>Reporte de Errores Léxicos</h2>

      <% if (erroresLexicos && erroresLexicos.length > 0) { %>
        <table id="tablaErrores">
          <thead>
            <tr>
              <th>#</th>
              <th>Fila</th>
              <th>columna</th>
              <th>lexema</th>
              <th>Descripción</th>
            </tr>
          </thead>
          <tbody>
            <% erroresLexicos.forEach((error, index) => { %>
              <tr>
                <td><%= index + 1 %></td>
                <td><%= error.fila %></td>
                <td><%= error.columna %></td>
                <td><%= error.lexema %></td>
                <td><%= error.token %></td>
              </tr>
            <% }) %>
          </tbody>
        </table>
      <% } else { %>
        <p>No se encontraron errores léxicos.</p>
      <% } %>
    </section>
  </main>
</body>
</html>
```

index.js

Se encarga de la carga del archivo, al igual que hacer la limpieza y la acción del botón analizar

Este script maneja la interacción del usuario con la interfaz. Sus funciones principales son:

- **Cargar archivo .plfp:** Lee el contenido del archivo seleccionado y lo coloca en el área de texto.
- **Limpiar el editor:** Borra el contenido del área de texto.
- **Guardar archivo:** Descarga el contenido actual del editor como un archivo .plfp.
- **Análisis léxico:**
 - Envía el texto al backend (/analyze) para su análisis. ○ Muestra los tokens válidos en una tabla.
 - En caso de errores, los envía al servidor para mostrar el reporte HTML.

```
document.addEventListener('DOMContentLoaded', () => {
  const fileInput = document.getElementById('fileInput');
  const archivoMenu = document.getElementById('archivoMenu');
  const editor = document.getElementById('editor');
  const formPensum = document.getElementById('formPensum');
  const inputPensum = document.getElementById('pensumContent');

  // Manejo del menú Archivo
  archivoMenu.addEventListener('change', () => {
    const opcion = archivoMenu.value;

    if (opcion === 'limpiar') {
      editor.value = '';
    } else if (opcion === 'cargar') {
      fileInput.click(); // abrir selector de archivos
    } else if (opcion === 'guardar') {
      const blob = new Blob([editor.value], { type: 'text/plain;charset=utf-8' });
      const a = document.createElement('a');
      a.href = URL.createObjectURL(blob);
      a.download = 'archivo.plfp';
      a.click();
    }

    archivoMenu.selectedIndex = 0; // reiniciar menú
  });

  // Cargar contenido del archivo seleccionado
  fileInput.addEventListener('change', (e) => {
    const file = e.target.files[0];

    if (!file) return;

    // Validar que sea archivo .plfp
    if (!file.name.toLowerCase().endsWith('.plfp')) {
      alert('Solo se permiten archivos con extensión .plfp');
      fileInput.value = ''; // limpiar selección
      return;
    }

    const reader = new FileReader();
    reader.onload = function (event) {
      editor.value = event.target.result;
    };
    reader.readAsText(file);
  });

  // Asignar el contenido del textarea al input oculto del botón Pensum
  formPensum.addEventListener('submit', function (e) {
    inputPensum.value = editor.value;
  });
});
```

index.ts

Configura y lanza el servidor express al igual que define middelwares estáticos y motor de vistas

```
import express from 'express';
import path from 'path';
import analyzeRouter from './routes/analyze.route';

const app = express();
const PORT = 3000;

// Configuración de vistas EJS
app.set('views', path.join(__dirname, '../views'));
app.set('view engine', 'ejs');

// Archivos estáticos
app.use(express.static(path.join(__dirname, '../public')));

// Middleware para procesar formularios
app.use(express.urlencoded({ extended: true }));
app.use(express.json());

// Rutas
app.use(analyzeRouter);

// Servidor
app.listen(PORT, () => {
  console.log(` Servidor corriendo en http://localhost:\${PORT}`);
});
```

analyze.route.ts

Recibe el texto para el análisis, renderiza el index.ejs y errores.ejs

```
import { Router } from 'express';
import { home, analyze, errorReport } from '../controllers/analyze.controller';
import { renderPensum } from '../controllers/analyze.controller';

const analyzeRouter = Router();

analyzeRouter.get('/', home);
analyzeRouter.post('/analyze', analyze);
analyzeRouter.post('/error-report', errorReport);
analyzeRouter.post('/pensum', renderPensum);
export default analyzeRouter;
```

analyze.controller.ts

Lleva el análisis y es el que llama al AFD. También gestiona los métodos de las rutas

```
import { Request, Response } from 'express';
import { LexicalAnalyzer } from '../Analyzer/LexicalAnalyzer';
import { Token } from '../Analyzer/Token';
import { extraerPensum } from '../Analyzer/PensumParser';

let lastLexicalErrors: { fila: number; columna: number; lexema: string; token: string }[] = [];

export const home = (_req: Request, res: Response) => {
  res.render('pages/index', {
    tokens: [],
    errors: [],
    codigo: '',
    contador: 0
  });
};

export const analyze = (req: Request, res: Response) => {
  const input = req.body.txtArea || '';
  const pensumExtraido = extraerPensum(input);
  const lexicalAnalyzer = new LexicalAnalyzer();
  const tokenList = lexicalAnalyzer.scanner(input);
  const rawErrorList = lexicalAnalyzer.getErrorList();

  lastLexicalErrors = rawErrorList.map((errorToken: Token) => ({
    fila: errorToken.getRow(),
    columna: errorToken.getColumn(),
    lexema: errorToken.getLexeme(),
    token: errorToken.getTypeTokenString()
  }));

  const tokensToSend = tokenList.map(token => ({
    fila: token.getRow(),
    columna: token.getColumn(),
    lexema: token.getLexeme(),
    token: token.getTypeTokenString()
  }));

  res.render('pages/index', {
    tokens: tokensToSend,
    errors: rawErrorList,
    codigo: input,
    contador: tokensToSend.length,
    carrera: pensumExtraido.carrera,
    semestres: pensumExtraido.semestres
  });
};

export const errorReport = (_req: Request, res: Response) => {
  res.render('pages/errores', {
    erroresLexicos: lastLexicalErrors
  });
};

export const renderPensum = (req: Request, res: Response) => {
  const input = req.body.txtArea || '';

  const { carrera, semestres } = extraerPensum(input);

  res.render('pages/pensum', {
    carrera,
    semestres
  });
};
```

Automata Finito Determinista AFD

